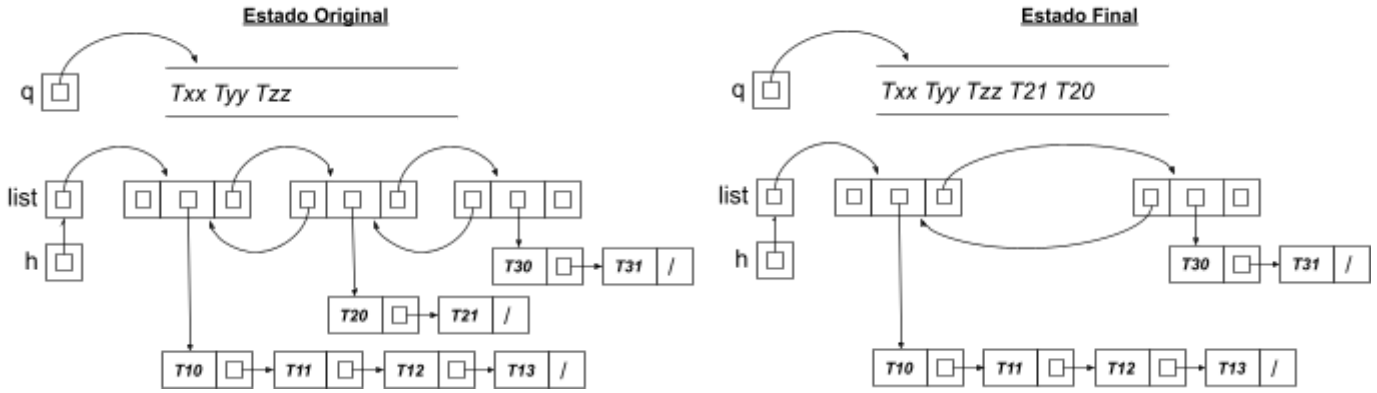
Nombre y apellido: _____

Ejercicios (puntaje)			Nota
1 (4 pts)	2 (2 pts)	3 (4 pts)	

CONSIDERACIONES: RESOLVER CADA EJERCICIO EN UNA HOJA DIFERENTE. LOS EJERCICIOS QUE NO ESTÉN CORRECTOS EN UN 50% NO SUMARÁN PUNTOS PARA LA NOTA FINAL. PARA CADA EJERCICIO DEBE **DEFINIR EL TIPO DE DATO** DE CADA TDA UTILIZADO. **EN TODOS LOS CASOS QUE UTILICE ESTRUCTURAS** QUE NO SEAN TDAs "ESTÁNDAR" **DEBE DEFINIR LOS STRUCTS**.

Ejercicio 1: Se tiene una lista con sublistas de tareas (TDA *Task*). El TDA *Task* se utiliza para representar tareas que se alojan en una *Queue* en un orden específico. Se solicita realizar una función, y todas las necesarias, para **encontrar la sublista más corta** (en caso de 2 o más mínimas, la más cercana al inicio de la lista), **eliminar el nodo de la lista, la sublista** y **agregar a la Queue todas sus tareas (*Task**) **en orden inverso al de la sublista**. La **lista** está implementada como una **lista dinámica doblemente enlazada** y la **sublista** está implementada como **lista dinámica simplemente enlazada**. Ejemplo:



Encabezado de la función principal: `void enqueue_task (dnode** h, queue* q);`

Ejercicio 2: Desarrollar una función recursiva que determine si los dígitos de un número natural están dispuestos de forma creciente. Por ejemplo: 2337, 356, 88, 459.

Ejercicio 3: Se representa una ciudad con matriz cuadrada de dimensión N x N donde cada celda es un punto geográfico. Cada punto puede ser de un tipo: Estación de autobuses (E), Zona congestionada (Z), Punto de interés (P) o vacío. Se desea **contabilizar la cantidad de puntos geográficos por tipo** (distintos de vacío) que hay en el camino más directo de un punto ***A(i,j)*** a un punto ***B(i,j)*** de la ciudad. Ejemplos:

